

Uso de Certificado en Sitio Web

Hugo Mauricio Romero Rodríguez
Miguel Angel Leal Perez
Miguel Angel Durazno Martinez
Omar Hernandez Cervantes

DSO1SV-25

Héctor Saldaña Benítez

Desarrollo Web Profesional

17 de julio de 2026
San Juan del Río, Qro.

SEAL

Despliegue local con Docker, Nginx y certificado HTTPS

Materia	Desarrollo Web Profesional
Proyecto	SEAL - Sistema de contratos inteligentes
Equipo	Hugo Mauricio Romero Rodriguez, Miguel Angel Leal Perez, Miguel Angel Durazno Martinez y Omar Hernandez Cervantes
Entorno	Despliegue local en macOS con Docker Desktop
Fecha	Julio de 2026

Documento técnico para explicar cómo se construyó y comprobó el despliegue local de SEAL usando Next.js, Express, Nginx, Docker Compose y un certificado generado con mkcert.

1. Objetivo de la Actividad

La actividad solicita investigar la forma de desplegar una aplicación web desarrollada con Next.js, colocar Nginx como proxy inverso e instalar un certificado. También menciona que la solución puede ejecutarse en contenedores Docker. Debido a que se trata de una práctica académica, se eligió realizar el despliegue en la computadora local, sin contratar un servidor y sin comprar un dominio.

El objetivo fue que SEAL dejara de ejecutarse solamente con los comandos de desarrollo y quedara organizado como una aplicación desplegada. Para conseguirlo, el frontend, el backend y Nginx se separaron en contenedores. El acceso se realiza mediante `https://localhost` y Nginx decide qué servicio debe atender cada petición.

2. Solución Seleccionada

Se eligió Docker Compose porque permite describir todos los servicios en un solo archivo y arrancarlos con el mismo comando. La solución utiliza tres contenedores: uno para Next.js, uno para Express y uno para Nginx. Los dos primeros permanecen dentro de la red privada de Docker y Nginx es el único que publica puertos hacia la computadora.

Para HTTPS se utilizó mkcert. Esta herramienta crea una autoridad certificadora local y genera certificados confiables para desarrollo. El certificado se instaló únicamente en la Mac utilizada para la práctica, por lo que no debe confundirse con un certificado público de Internet.

3. Arquitectura del Despliegue

La arquitectura mantiene una sola entrada para toda la aplicación. El navegador se comunica con Nginx por HTTPS. Después, Nginx reenvía la petición al frontend o al backend de acuerdo con la ruta solicitada.

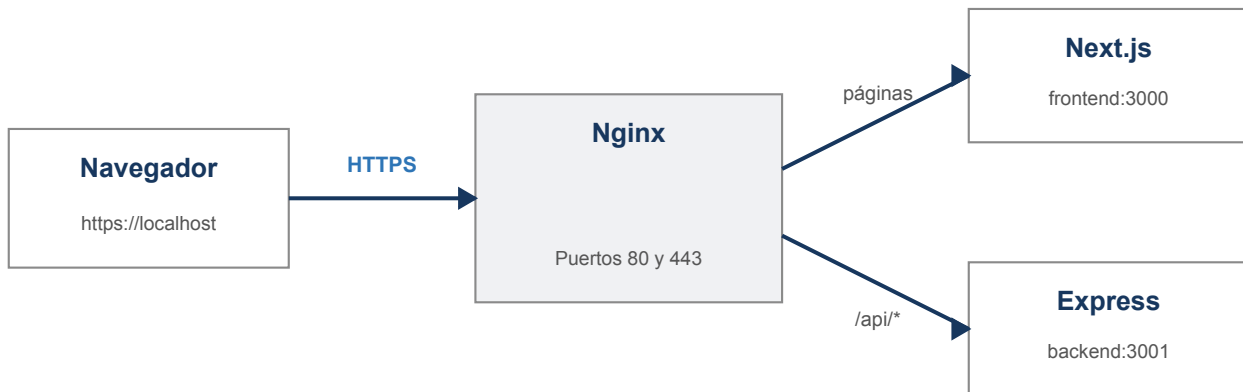


Figura 1. Flujo de una petición dentro del despliegue local de SEAL.

El usuario no necesita entrar directamente a los puertos 3000 o 3001. Esto hace que la dirección sea más sencilla y permite centralizar el certificado en Nginx. Las rutas de páginas, como /login o /contratos, se envían a Next.js. Las rutas que comienzan con /api se envían al backend Express.

Dirección solicitada	Servicio que responde	Destino interno
https://localhost/login	Next.js	frontend:3000/login
https://localhost/contratos	Next.js	frontend:3000/contratos
https://localhost/api/health	Express	backend:3001/health
https://localhost/api/auth/login	Express	backend:3001/auth/login

4. Componentes Utilizados

Next.js

Es el frontend de SEAL. Se configuró con output: standalone para producir una versión autocontenida que puede ejecutarse en una imagen Docker más pequeña. El contenedor escucha en el puerto interno 3000.

Express

Es el backend que maneja autenticación, contratos, clientes, firmas y almacenamiento. El contenedor escucha en el puerto interno 3001 e incluye Chromium porque SEAL lo utiliza para generar archivos PDF.

Nginx

Es el proxy inverso y la puerta de entrada. Publica los puertos 80 y 443, redirige HTTP hacia HTTPS, presenta el certificado y reenvía cada solicitud al servicio correspondiente.

5. Configuración con Docker

Se crearon archivos Dockerfile independientes para el frontend y el backend. También se creó compose.local.yaml para coordinar la construcción de imágenes, la red interna, el volumen de almacenamiento, las comprobaciones de salud y el arranque de Nginx.

Archivo	Función dentro del proyecto
Front/Dockerfile	Instala dependencias, compila Next.js y ejecuta la salida standalone.
Back/Dockerfile	Instala dependencias de producción, Chromium y ejecuta Express.
compose.local.yaml	Define frontend, backend, Nginx, red, volumen y healthchecks.
deploy/nginx/local.conf	Contiene las reglas HTTPS y proxy_pass.
deploy/generate-local-cert.sh	Instala la CA local y genera el certificado de localhost.

5.1 Construcción del Frontend

El Dockerfile del frontend usa varias etapas. Primero instala las dependencias con npm ci; después ejecuta npm run build; finalmente copia sólo los archivos necesarios para ejecutar el servidor. Este proceso evita incluir herramientas de desarrollo innecesarias en la imagen final.

```
ARG NEXT_PUBLIC_API_URL=/api
RUN npm run build
CMD ["node", "server.js"]
```

5.2 Construcción del Backend

El backend instala únicamente dependencias de producción. También instala Chromium y fuentes del sistema para conservar la generación de contratos en PDF. El proceso se ejecuta con un usuario sin privilegios y utiliza tini para manejar correctamente el proceso de Node.js.

```
ENV PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium
ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["node", "src/server.js"]
```

5.3 Docker Compose

Docker Compose crea una red llamada seal-local_default. Dentro de esa red, Nginx puede encontrar al frontend usando el nombre frontend y al backend usando backend. No es necesario conocer las direcciones IP internas, ya que Docker resuelve los nombres automáticamente.

También se creó el volumen backend_local_storage para conservar archivos generados por SEAL aunque los contenedores se vuelvan a crear.

6. Nginx como Proxy Inverso

Nginx se coloca entre el navegador y los servicios de la aplicación. Por esta razón se le llama proxy inverso: recibe la solicitud en nombre de los servidores internos y después la dirige al destino correcto.

La primera regla redirige cualquier solicitud HTTP hacia HTTPS. Esto evita que la aplicación se use accidentalmente sin cifrado.

```
server {
  listen 80;
  return 301 https://$host$request_uri;
}
```

Para las páginas normales, Nginx utiliza el servicio frontend. Los encabezados conservan información sobre el host original y el protocolo usado por el navegador.

```
location / {
  proxy_pass http://frontend:3000;
  proxy_set_header Host $host;
  proxy_set_header X-Forwarded-Proto $scheme;
}
```

Para las rutas del API se utiliza el servicio backend. La diagonal final de proxy_pass hace que el prefijo /api se retire antes de entregar la petición a Express. Por ejemplo, /api/health se convierte en /health.

```
location /api/ {
  proxy_pass http://backend:3001/;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

6.1 Puertos Publicados

Servicio	Puerto interno	¿Visible desde la Mac?
Next.js	3000	No. Sólo se usa dentro de Docker.
Express	3001	No. Sólo se usa dentro de Docker.
Nginx	80 y 443	Sí. Es la entrada oficial de SEAL.

La salida de docker compose ps confirmó esta configuración. Frontend y backend mostraron únicamente 3000/tcp y 3001/tcp, mientras que Nginx mostró la publicación de 0.0.0.0:80 y 0.0.0.0:443.

7. Instalación del Certificado

Para activar HTTPS sin contratar infraestructura se utilizó mkcert. Primero se instaló la herramienta mediante Homebrew. Después, el script del proyecto ejecutó `mkcert -install`, lo que creó una autoridad certificadora local y la agregó al almacén de confianza de macOS.

```
brew install mkcert
chmod +x deploy/generate-local-cert.sh
./deploy/generate-local-cert.sh
```

El script generó dos archivos dentro de `deploy/certs`:

- `seal-local.pem`: certificado público que Nginx presenta al navegador.
- `seal-local-key.pem`: llave privada utilizada para establecer la conexión cifrada.

El certificado fue emitido para `localhost`, `127.0.0.1` y `::1`. La revisión con OpenSSL confirmó que fue firmado por `mkcert development CA` y que es válido del 17 de julio de 2026 al 17 de octubre de 2028.

```
Titular: mkcert development certificate
Emisor: mkcert development CA
Válido hasta: 17 de octubre de 2028
```

7.1 Diferencia con un Certificado Público

El certificado de `mkcert` es apropiado para desarrollo porque la propia Mac confía en la autoridad que lo firmó. Sin embargo, otra computadora o un teléfono no confiarán automáticamente en él. Si SEAL se publicara en Internet, se usaría un dominio y un certificado emitido por una autoridad pública, por ejemplo Let's Encrypt.

Característica	Despliegue realizado	Despliegue público
Dirección	<code>https://localhost</code>	<code>https://seal.midominio.com</code>
Certificado	<code>mkcert</code>	Let's Encrypt
Costo	Sin costo	El certificado es gratuito; el servidor o dominio pueden generar costo
Confianza	Sólo en equipos donde se instale la CA	Navegadores y teléfonos confían automáticamente

La llave privada y los certificados locales se excluyeron del repositorio mediante `.gitignore`. Tampoco debe compartirse `rootCA-key.pem`, ya que representa la llave privada de la autoridad local creada por `mkcert`.

8. Procedimiento Ejecutado

El despliegue se realizó desde la raíz del proyecto SEAL. Los siguientes pasos resumen el procedimiento completo que se ejecutó en la Mac.

- 1 Instalar mkcert con Homebrew.
- 2 Entrar a la carpeta del proyecto SEAL.
- 3 Dar permiso de ejecución al script generate-local-cert.sh.
- 4 Ejecutar el script para instalar la CA y generar el certificado.
- 5 Construir y levantar los contenedores con Docker Compose.
- 6 Revisar que frontend y backend aparezcan como healthy.
- 7 Abrir <https://localhost/login> y comprobar la aplicación.
- 8 Probar <https://localhost/api/health> para verificar el proxy hacia Express.

Comandos utilizados

```
cd ".../Desarrollo Web Profesional/3er Parcial/Seal"
docker compose -f compose.local.yaml up -d --build
docker compose -f compose.local.yaml ps
curl https://localhost/api/health
```

El comando `up -d --build` construyó las imágenes, creó la red interna, creó el volumen de almacenamiento e inició los contenedores. La opción `-d` permitió que los servicios continuaran en segundo plano.

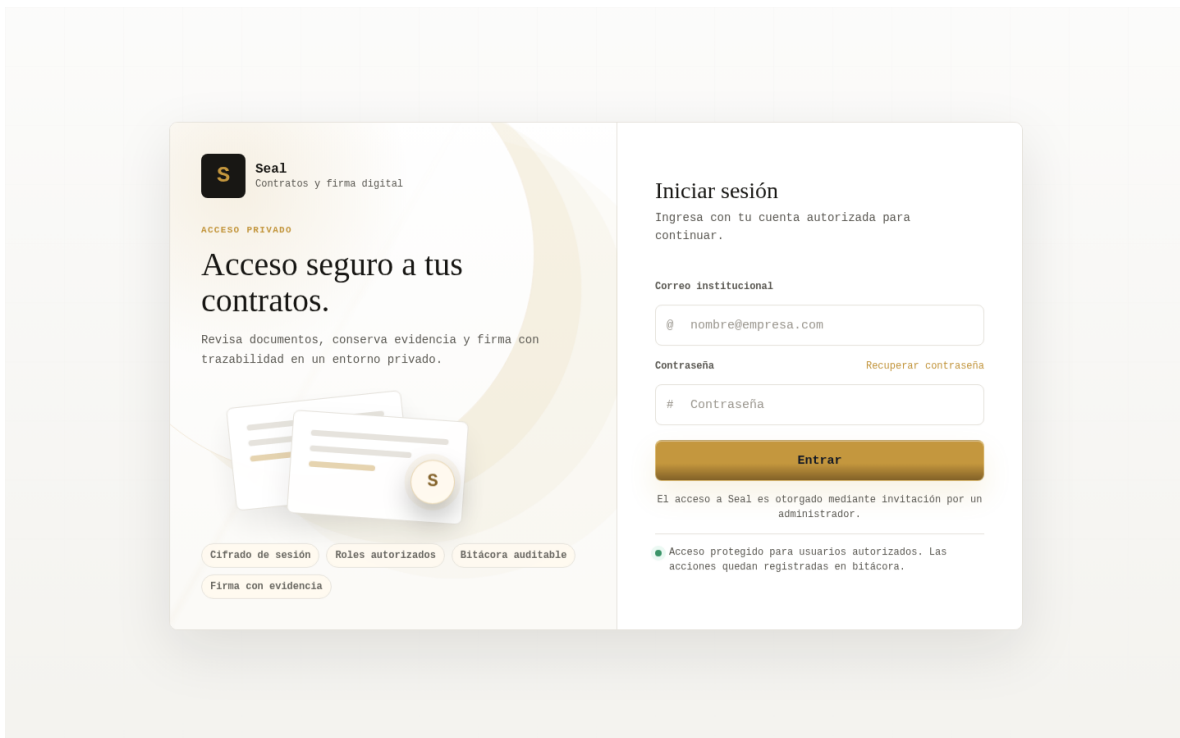


Figura 2. Pantalla de inicio de sesión servida desde el despliegue local de SEAL.

9. Pruebas y Evidencias

Se realizaron varias comprobaciones para asegurar que el resultado no fuera únicamente visual. Las pruebas verificaron el estado de los contenedores, la comunicación por HTTPS, el funcionamiento del proxy y la validez del certificado.

9.1 Estado de los Contenedores

```
frontend Up (healthy) 3000/tcp
backend Up (healthy) 3001/tcp
nginx Up 0.0.0.0:80->80, 0.0.0.0:443->443
```

Los healthchecks de Docker confirmaron que Next.js y Express respondían antes de iniciar Nginx. Esto reduce la posibilidad de que el proxy empiece a recibir solicitudes cuando los servicios todavía no están listos.

9.2 Prueba del API

```
$ curl https://localhost/api/health
{"ok":true}
```

Esta respuesta recorrió el flujo completo: el navegador o curl validó el certificado, Nginx recibió la conexión HTTPS, retiró el prefijo /api, reenvió /health al backend y Express devolvió el JSON.

9.3 Prueba del Certificado

La conexión se realizó sin utilizar curl -k. Esta opción normalmente se utiliza para ignorar errores TLS, por lo que no necesitarla demuestra que la CA local está instalada y que el certificado presentado por Nginx es confiable en macOS.

10. Medidas de Seguridad

- Los archivos Back/.env y firebase-admin.json permanecen fuera de Git y de las imágenes.
- La llave privada del certificado está en deploy/certs, carpeta excluida por .gitignore.
- Frontend y backend no exponen directamente sus puertos a la computadora.
- Los procesos de Node se ejecutan con usuarios sin privilegios dentro de los contenedores.
- Nginx centraliza HTTPS y redirige las solicitudes HTTP al puerto seguro 443.

Comandos de operación

```
docker compose -f compose.local.yaml logs -f
docker compose -f compose.local.yaml up -d --build
docker compose -f compose.local.yaml down
```


11. Resultado Obtenido

SEAL quedó desplegado localmente mediante tres contenedores. Next.js sirve la interfaz, Express atiende las operaciones del API y Nginx funciona como punto de entrada. El acceso principal se realiza con `https://localhost` y las solicitudes del API utilizan el mismo origen mediante el prefijo `/api`.

El certificado generado con `mkcert` fue instalado correctamente y el navegador lo acepta en la Mac donde se realizó la práctica. Esto permitió demostrar el uso de HTTPS sin pagar un servidor, un servicio de alojamiento o un dominio.

La implementación también deja preparada una variante para un despliegue público. En ese escenario se conservarían los contenedores y las reglas de Nginx, pero se reemplazaría el certificado local por uno emitido por Let's Encrypt y el sitio se publicaría con un dominio real.

12. Conclusión

La actividad permitió comprender que desplegar una aplicación no consiste solamente en ejecutar `npm run dev`. En un despliegue se preparan versiones de producción, se organizan los servicios, se definen puertos, se protege la comunicación y se comprueba que cada componente responda correctamente.

Nginx resultó útil porque concentra la entrada de la aplicación. El navegador se comunica únicamente con Nginx y éste decide si la solicitud corresponde al frontend o al backend. Además, el certificado se instala en un solo lugar, en vez de configurar HTTPS por separado en Next.js y Express.

Docker facilitó la práctica porque aisló cada servicio y permitió levantar toda la solución con un comando. Gracias a esta estructura, el mismo proyecto puede reproducirse en otra computadora que tenga Docker, siempre que se configuren las variables de entorno y credenciales necesarias.

Finalmente, `mkcert` permitió cubrir el requisito del certificado dentro de un entorno académico y sin costo. Se debe recordar que es una herramienta de desarrollo. Para publicar SEAL en Internet se necesitaría un dominio, un servidor accesible y un certificado público.

Referencias Bibliográficas

- Docker Docs. (s. f.). Networking in Compose. <https://docs.docker.com/compose/how-tos/networking/>
- FiloSottile. (s. f.). mkcert: A simple zero-config tool to make locally trusted development certificates. <https://github.com/FiloSottile/mkcert>
- Let's Encrypt. (s. f.). Challenge Types. <https://letsencrypt.org/docs/challenge-types/>
- Nginx. (s. f.). Configuring HTTPS servers. https://nginx.org/en/docs/http/configuring_https_servers.html
- Next.js. (s. f.). Self-Hosting. <https://nextjs.org/docs/app/guides/self-hosting>